

Reflexión Individual sobre los Principios SOLID

Entrega individual — Brightspace

Estudiante	Josue Riera (Backend Developer)
Categoría de grupo	Semana 1 - Autoinscripcion
Nombre de grupo	S1 1 - Martes
Repositorio	https://github.com/Lyriom/calidad32

1. ¿Cuál fue el principio más desafiante de aplicar? ¿Por qué?

Para mí el más desafiante fue la **Inversión de Dependencias (DIP)**, y no tanto por su definición sino porque me obligó a desaprender una costumbre muy arraigada: yo siempre creaba lo que necesitaba con `new` dentro de la misma clase que lo usaba. En nuestro ejemplo, un `Interruptor` 'ingenuo' habría tenido un `new Bombilla()` en su interior, y eso parecía totalmente inofensivo. El problema se hizo visible recién cuando sumamos el `Ventilador` y la `Cafetera`: cada dispositivo nuevo me empujaba a abrir y editar el interruptor una y otra vez. Al reescribirlo para que recibiera un `Dispositivo` por el constructor, el interruptor dejó de conocer cualquier detalle concreto, y entendí que invertir la dependencia no era una complicación, sino justo lo que da flexibilidad.

2. ¿Cómo crees que SOLID mejora el diseño de software?

Pienso que SOLID mejora el diseño porque convierte el cambio en algo seguro en lugar de arriesgado. Al apoyarse en abstracciones y en responsabilidades bien delimitadas, puedo intervenir una pieza del sistema con la tranquilidad de que el resto no se va a venir abajo. Para mí, que trabajo en backend, el beneficio más concreto es que el código se vuelve testeable: puedo sustituir una dependencia real por una falsa en las pruebas sin tocar la lógica principal.

3. ¿Qué principio SOLID aplicarías en futuros o actuales proyectos?

En lo que viene aplicaría **DIP** de forma deliberada en cualquier punto donde dependamos de un servicio externo —pasarelas de pago, envío de correos o SMS, almacenamiento en la nube—. La regla que me llevo es simple: no instanciar al proveedor directamente, sino programar contra una interfaz e inyectar la implementación desde afuera. Así, cambiar de proveedor se reduce a cambiar una clase y no a reescribir el núcleo del sistema.

